Studio Nx

P1-13 프로젝트 API — 함수 설명 합본 (백엔드 56~62)

2026-10-08

목차

1. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/56_project_schema_request.md
2. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/57_project_service_membership.md
3. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/58_project_service_list.md
4. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/59_project_service_create.md
5. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/60_project_service_update.md
6. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/61_project_service_delete.md
7. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/62_projects_routes.md

Backend(FastAPI/Python) 56 — `ProjectCreateRequest` · `ProjectUpdateRequest` (프로젝트 요청 스키마)

항목	내용
파일	`studionx/backend/app/schemas/project.py`
작업	P1-13 프로젝트 API — 만들기 · 수정 요청 본문 검사 · PRD 01 FR-122 · FR-124
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	55 `retention.main()` · 57 `get membership` · `require role`

1. 하는 일

새 프로젝트 모달 · Edit details 모달이 보낸 값을 **저장하기 전에** 검사한다. 틀리면 422 `VALIDATION_ERROR` (P1-03 처리기).

보낸 값	결과
`{"title": " My Song "}`	통과 — 제목 `"My Song"`, 설명 없음, BPM 120, 4/4
제목 `""` · `" "` · 101자	422
설명 `" "`	통과 — 설명 **없음**(None)
박자 `7/8` (`time_sig_num=7, time_sig_den=8`)	통과
박자 아래 숫자 `3` · `12` · `64`	422 — 1 · 2 · 4 · 8 · 16 · 32 만
수정 `{"title": null}`	422 — 제목은 비울 수 없음
수정 `{}`	통과 — 아무것도 바꾸지 않음
수정 `{"description": ""}` 또는 `null`	통과 — 설명 지움

2. 언제 쓰나

요청	스키마	화면
`POST /api/v1/projects`	`ProjectCreateRequest`	myProject.html 새 프로젝트 모달 (P1-14)
`PATCH /api/v1/projects/{id}`	`ProjectUpdateRequest`	Edit details 모달 (P1-14)

FastAPI 가 라우터 인자의 타입을 보고 자동으로 검사한다. 서비스(58 · 59)는 이미 검사 · 정리된 값을 받는다.

3. 코드

import · 상수 (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from enum import StrEnum
from typing import Annotated
```

```
from pydantic import BaseModel, Field, StringConstraints, field_validator

PROJECT_PAGE_SIZE_MAX = 50  # 한 번에 받을 수 있는 최대 개수 (화면은 10)
PROJECT_TITLE_MAX = 100  # 화면 #new-title · #edit-title maxlength 와 같음
PROJECT_DESCRIPTION_MAX = 500  # 화면 #new-description · #edit-description maxlength 와 같음
TIME_SIG_DENOMINATORS = (1, 2, 4, 8, 16, 32)  # 박자 아래 숫자 (음표 길이)

ProjectTitle = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=PROJECT_TITLE_MAX)
]
ProjectDescription = Annotated[
    str, StringConstraints(strip_whitespace=True, max_length=PROJECT_DESCRIPTION_MAX)
]
```

`Annotated` · `StringConstraints` · `field_validator` 와 상수 · 타입 4개를 이번에 추가했다.

만들기

```
class ProjectCreateRequest(BaseModel):
    title: ProjectTitle
    description: ProjectDescription | None = None
    bpm: Decimal = Field(default=Decimal("120"), ge=20, le=999, decimal_places=2)
    time_sig_num: int = Field(default=4, ge=1, le=32)
    time_sig_den: int = Field(default=4)

    @field_validator("description")
    @classmethod
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None  # 공백만 친 설명 → "" → None

    @field_validator("time_sig_den")
    @classmethod
    def _check_den(cls, value: int) -> int:
        if value not in TIME_SIG_DENOMINATORS:
            raise ValueError("must be one of 1, 2, 4, 8, 16, 32")
        return value
```

`bpm` · `time_sig_num` 줄은 템플릿 그대로.

수정

```
class ProjectUpdateRequest(BaseModel):
    title: ProjectTitle | None = None
    description: ProjectDescription | None = None

    @field_validator("title")
    @classmethod
    def _title_not_null(cls, value: str | None) -> str:
        # 보내지 않으면 이 검사는 돌지 않음 (기본값은 검사하지 않음) → null 을 직접 보낸 경우만
        if value is None:
            raise ValueError("title cannot be empty")
        return value

    @field_validator("description")
    @classmethod
```

```
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None
```

문법 풀이

import 문법	뜻
`from typing import Annotated`	타입에 검사 규칙을 붙여 새 이름으로 (`ProjectTitle`)
`from pydantic import StringConstraints, field_validator`	글자 규칙(공백 제거 · 길이) / 칸 하나를 직접 검사하는 함수

식	뜻
`StringConstraints(strip_whitespace=True, min_length=1, …)`	앞뒤 공백을 **먼저** 지우고 길이를 잼
`ProjectTitle | None = None`	제목 또는 null, 안 보내면 None
`@field_validator("time_sig_den")` + `@classmethod`	그 칸의 기본 검사가 끝난 뒤 불리는 함수 (pydantic 규칙상 classmethod)
`raise ValueError(...)`	검사 실패 → pydantic 이 422 로 바꿈
`value or None`	`""` 처럼 빈 값이면 None, 아니면 그대로
`model_dump(exclude_unset=True)`	(59 에서 씀) **보낸 칸만** 꺼냄 — 보내지 않은 칸은 바꾸지 않으려고

4. 흐름

1. JSON 을 받아 칸마다 타입 · 범위 검사 (BPM 20~999, 소수 둘째 자리까지, 박자 위 1~32).
2. 제목 · 설명은 앞뒤 공백을 지운 뒤 길이 검사.
3. `field_validator` — 빈 설명은 None, 박자 아래 숫자 확인, (수정) 제목 null 거부.
4. 하나라도 틀리면 422, 아니면 서비스로.

5. 왜 이렇게

선택	이유
제목을 **공백 제거 후** 검사	템플릿(`min_length=1`)대로면 공백 한 칸짜리 제목이 저장된다 — 닉네임(37) · 문의(50)와 같은 빈틈. 서비스의 `data.title.strip()`(템플릿)은 필요 없어짐
빈 설명 → None	칸을 비우는 것 = 설명 없음. `""` 와 `NULL` 두 가지가 섞이지 않게. Edit details 에서 설명을 지우면 화면은 빈 칸을 보냄
박자 아래 숫자는 6개만	음표 길이(온 · 2분 · 4분 · 8분 · 16분 · 32분)만 박자가 된다 (템플릿 TODO). 화면은 4/4 · 3/4 · 6/8 · 7/8 만 고르지만 서버도 막음
수정에서 제목 null 거부를 **스키마**에서	템플릿은 서비스에서 422 를 내라고 했지만, 스키마에서 막으면 다른 검사 오류와 같은 모양(`VALIDATION_ERROR` + `details`)으로 나간다
길이 상수를 화면과 같게	화면 `maxlength` 100 · 500 과 서버가 어긋나지 않게 (문의 50 과 같음)

6. 시험 (`tests/test_project_schemas.py` 새 파일 · 33개)

시험	확인한 것
기본값	제목만 → 설명 None · BPM 120 · 4/4
공백	제목 · 설명 앞뒤 공백 제거, 공백 뒤 100자 제목 통과
틀린 제목 4가지	`""` · 공백 · 줄바꿈만 · 101자 → 422
설명	빈 칸 · 공백 → None, 501자 → 422
박자	아래 1 · 2 · 4 · 8 · 16 · 32 통과 / 0 · 3 · 5 · 12 · 64 · −4 → 422, 위 0 · 33 → 422
BPM	19.99 · 999.01 · 소수 셋째 자리 → 422
수정	빈 요청 → 바꿀 것 없음, 제목 null · 빈 · 101자 → 422, 공백 제목 정리 · 빈 설명 → None

전체 시험 438개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 57 — `get_membership()` · `require_role()` (프로젝트 권한 검사)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 모든 프로젝트 API 의 첫 검사 · PRD 01 §5 권한 규칙
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	56 프로젝트 요청 스키마 · 58 `to_item` · `list_projects`

1. 하는 일

- `get_membership`: 프로젝트 번호로 프로젝트와 **내 멤버 행**을 함께 찾는다. 접근할 수 없으면 404.
- `require_role`: 그 멤버 행의 역할이 허용 목록에 있는지 본다. 없으면 403.

상황	결과
내가 멤버인 프로젝트	`(project, member)` 돌려줌
없는 번호	404 `"Project not found."`
남의 프로젝트 (내가 멤버가 아님)	404 `"Project not found."` — 403 아님 (사용자 결정 A)
삭제된 프로젝트 (`deleted_at` 있음)	404 — 주인이어도
VIEWER 가 수정 · EDITOR 가 삭제	403 `"You don't have permission."` (`require_role`)

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /projects/{id}` 등 프로젝트 하나를 다루는 라우터 (62)	`get_membership` — 멤버면 누구나 조회
`update_project` (60)	`require_role(member, "OWNER", "EDITOR")`
`delete_project` (61)	`require_role(member, "OWNER")`
시퀀서 · 협업 API (Phase 2 · 6)	같은 두 함수를 그대로 씀

역할(`project_members.role`)은 OWNER · EDITOR · VIEWER 세 가지. Phase 1 에서는 만든 사람의 OWNER 한 행만 생긴다 (초대는 Phase 6).

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, User
from app.schemas.project import (
```

```
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`AppError` · `ErrorCode` 줄을 이번에 추가했다. 두 함수는 `Session` · `AppError` · `ErrorCode` · `Project` · `ProjectMember` · `User` 를 쓴다.

함수

```
def get_membership(db: Session, project_id: int, user: User) -> tuple[Project, ProjectMember]:
    project = db.get(Project, project_id)
    member = None
    if project is not None and project.deleted_at is None:
        member = db.get(ProjectMember, (project_id, user.user_id))
    if member is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Project not found.")
    return project, member


def require_role(member: ProjectMember, *roles: str) -> None:
    if member.role not in roles:
        raise AppError(ErrorCode.FORBIDDEN, 403, "You don't have permission.")
```

문법 풀이

import 문법	뜻
`from app.core.errors import AppError, ErrorCode`	우리 오류 클래스 · 오류 코드 목록 (P1-03). `AppError` 를 던지면 처리기가 `{"error": {...}}` JSON 으로 바꿈

식	뜻
`db.get(Project, project_id)`	기본 키로 한 행 찾기. 없으면 `None`
`db.get(ProjectMember, (project_id, user.user_id))`	기본 키가 두 칸(복합 키)이면 **튜플**로 넘김
`-> tuple[Project, ProjectMember]`	두 값을 함께 돌려준다는 표시. 부르는 쪽은 `project, member = get_membership(...)`
`*roles: str`	인자를 몇 개든 받아 튜플로 모음. `require_role(m, "OWNER", "EDITOR")` → `roles == ("OWNER", "EDITOR")`
`member.role not in roles`	역할은 `MemberRole`(StrEnum) 이라 문자열 `"OWNER"` 와 같다고 비교됨

4. 흐름

1. 번호로 프로젝트를 찾는다.
2. 있고 삭제되지 않았을 때만 `(프로젝트 번호, 내 회원 번호)` 로 멤버 행을 찾는다.
3. 멤버 행이 없으면 — 이유(없음 · 삭제됨 · 남의 것)와 상관없이 — 404.
4. (수정 · 삭제일 때) `require_role` 로 역할 확인 → 모자라면 403.

5. 왜 이렇게

선택	이유
남의 프로젝트도 **404**	403 이면 "그 번호의 프로젝트는 있다"는 것이 드러난다. 번호를 1, 2, 3... 넣어 보며 남의 프로젝트가 있는지 알아낼 수 없게 (사용자 결정 A, 템플릿 권장)
세 경우를 `member is None` 하나로	없음 · 삭제됨 · 남의 것이 모두 같은 응답이어야 하므로 오류를 한 곳에서만 던짐
삭제된 프로젝트는 멤버 행을 찾지 않음	쓸데없는 쿼리를 줄이고, 주인이어도 삭제된 것은 못 보게 (소프트 삭제 61)
역할 부족은 **403**	이미 멤버라 프로젝트가 있다는 것을 아는 사람 → 숨길 것이 없고, "권한 없음"이 더 정확
두 함수로 나눔	조회는 멤버면 충분, 수정 · 삭제만 역할을 봄 — 필요한 곳에서만 `require_role` 을 한 줄 더 부름

6. 시험 (`tests/test_project_service.py` 새 파일 · 12개)

시험	확인한 것
주인	`(project, member)`, 역할 OWNER
EDITOR 멤버	멤버 행을 찾아 역할 EDITOR
없는 번호	404 `NOT_FOUND` "Project not found."
남의 프로젝트	404 (403 아님)
삭제된 프로젝트	주인이어도 404
`require_role` 통과 4가지	OWNER→OWNER, OWNER·EDITOR 에 OWNER · EDITOR, 셋 다 허용에 VIEWER
`require_role` 거부 3가지	EDITOR→OWNER 만, VIEWER→OWNER 만, VIEWER→OWNER·EDITOR → 403 `FORBIDDEN`

전체 시험 450개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 58 — `to_item()` · `list_projects()` (내 프로젝트 목록)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 목록 · 검색 · 정렬 · 페이지 · PRD 01 FR-123 · FR-130
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	57 `get_membership` · `require_role` · 59 `create_project`

1. 하는 일

- **`to_item`**: DB 의 `Project` 한 행을 화면이 쓰는 모양 `ProjectItem` 으로 바꾼다. 프로젝트 칸에 더해 세 칸을 계산해 넣는다.
- **`list_projects`**: 내가 멤버인 프로젝트를 검색 · 정렬 · 페이지로 잘라 `{items, total, page, size}` 로 돌려준다.

계산하는 칸	뜻	화면 (myProject.html)
`my_role`	이 프로젝트에서 내 역할 (OWNER · EDITOR · VIEWER)	카드의 역할 표시, 삭제 버튼은 OWNER 만
`member_count`	멤버 수	카드의 멤버 수
`opened`	**내가** 시퀀서에서 연 적이 있는지 (`project_sessions` 행이 있는지)	한 번도 안 열었으면 Not opened 표시 (FR-130)

요청 예	결과
`GET /projects`	최근 수정순, 1쪽 10개
`?sort=title`	이름순 (대소문자 무시: `alpha` · `Beta` · `Gamma`)
`?sort=created`	만든 순 (최근 것 먼저)
`?query=alp`	제목에 `alp` 가 들어간 것 (대소문자 무시)
`?query=%`	제목에 **글자 `%`** 가 들어간 것만 (모든 프로젝트가 아님)
`?page=3` (2쪽까지만 있음)	`items` 빈 목록, `total` 은 그대로

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /api/v1/projects` 라우터 (62)	`list_projects`
`GET` · `POST` · `PATCH /projects/...` 응답 (62 · 59 · 60)	`to_item(db, project, member)` — 세 칸을 직접 DB 에 물어봄
`list_projects` 안	`to_item(..., member_count, opened)` — 미리 모아 계산한 값을 넘김
화면	myProject.html `loadProjects` · `renderProjects` (P1-14)

`project_sessions` 는 시퀀서를 열 때마다 생기는 작업 기록 (기록은 Phase 2). 지금은 행이 없으니 모든 프로젝트가 `opened=false` 로 나온다.

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from sqlalchemy import func, select` 줄과 `ProjectSession` 을 이번에 추가했다. 두 함수는 `func` · `select` · `Session` · `Project` · `ProjectMember` · `ProjectSession` · `User` · `ProjectItem` · `ProjectListResponse` · `ProjectSort` 를 쓴다.

함수 (도움 함수 `_escape_like` · 정렬 표 `SORT_ORDER` 포함)

```
def to_item(
    db: Session,
    project: Project,
    member: ProjectMember,
    member_count: int | None = None,
    opened: bool | None = None,
) -> ProjectItem:
    if member_count is None:
        member_count = db.scalar(
            select(func.count())
            .select_from(ProjectMember)
            .where(ProjectMember.project_id == project.project_id)
        )
    if opened is None:
        session_id = db.scalar(
            select(ProjectSession.session_id)
            .where(
                ProjectSession.project_id == project.project_id,
                ProjectSession.user_id == member.user_id,
            )
            .limit(1)
        )
        opened = session_id is not None
    return ProjectItem(
        project_id=project.project_id,
        title=project.title,
        description=project.description,
        bpm=project.bpm,
```

```
        time_sig_num=project.time_sig_num,
        time_sig_den=project.time_sig_den,
        music_key=project.music_key,
        created_at=project.created_at,
        updated_at=project.updated_at,
        my_role=member.role,
        member_count=member_count,
        opened=opened,
    )


def _escape_like(text: str) -> str:
    return text.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")


SORT_ORDER = {
    ProjectSort.UPDATED: Project.updated_at.desc(),
    ProjectSort.TITLE: Project.title.asc(),
    ProjectSort.CREATED: Project.created_at.desc(),
}


def list_projects(
    db: Session,
    user: User,
    query: str,
    sort: ProjectSort,
    page: int,
    size: int,
) -> ProjectListResponse:
    conditions = [ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None)]
    if query:
        conditions.append(Project.title.ilike(f"%{_escape_like(query)}%", escape="\\"))

    total = db.scalar(
        select(func.count())
        .select_from(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
    )
    rows = db.execute(
        select(Project, ProjectMember)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
        .order_by(SORT_ORDER[sort], Project.project_id.desc())
        .offset((page - 1) * size)
        .limit(size)
    ).all()

    # 이 페이지 프로젝트들의 멤버 수 · 내가 연 적 있는지를 쿼리 두 번으로 모아 계산
    ids = [project.project_id for project, _ in rows]
    counts = {}
    opened_ids = set()
    if ids:
        counts = dict(
            db.execute(
                select(ProjectMember.project_id, func.count())
```

```
                .where(ProjectMember.project_id.in_(ids))
                .group_by(ProjectMember.project_id)
            ).all()
        )
        opened_ids = set(
            db.scalars(
                select(ProjectSession.project_id).where(
                    ProjectSession.project_id.in_(ids), ProjectSession.user_id == user.user_id
                )
            )
        )

    items = [
        to_item(db, project, member, counts[project.project_id], project.project_id in opened_ids)
        for project, member in rows
    ]
    return ProjectListResponse(items=items, total=total, page=page, size=size)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import func, select`	`select(...)` = SELECT 문 만들기, `func.count()` = SQL `COUNT(*)`

식	뜻
`member_count: int | None = None`	안 넘기면 None → 함수 안에서 DB 에 물어봄
`db.scalar(select(...))`	결과의 **첫 줄 첫 칸** 하나만 (개수 · 번호 하나)
`.limit(1)` + `is not None`	행이 **하나라도 있는지**만 확인 (다 세지 않음)
`SORT_ORDER[sort]`	정렬 종류 → `ORDER BY` 식. `if/elif` 세 줄 대신 표로
`.where(*conditions)`	목록의 조건을 모두 AND 로 (`*` = 목록을 풀어 인자 여러 개로)
`Project.title.ilike(..., escape="\\")`	대소문자 무시 `LIKE`. 검색어 안의 `\` 뒤 글자는 특수 문자가 아니라 글자 그대로
`.offset((page - 1) * size).limit(size)`	3쪽 · 10개 → 앞 20개 건너뛰고 10개
`db.execute(select(Project, ProjectMember)...).all()`	한 줄에 `(project, member)` 두 객체
`.in_(ids)` · `.group_by(...)`	`WHERE project_id IN (...)` · 프로젝트별로 묶어 세기
`dict(rows)`	`[(5, 2), (8, 1)]` → `{5: 2, 8: 1}` (프로젝트 번호 → 멤버 수)
`project.project_id in opened_ids`	집합(set)에 있는지 → True / False

4. 흐름

1. 조건 만들기: 내가 멤버 · 삭제 안 됨 (+ 검색어가 있으면 제목 LIKE).
2. 같은 조건으로 **전체 개수**(`total`)를 센다.
3. 정렬 → 같은 값이면 `project_id` 큰 것 먼저 → 해당 쪽만 잘라 `(project, member)` 를 가져온다.
4. 이 쪽의 프로젝트 번호들로 **쿼리 두 번**: 멤버 수(묶어 세기), 내가 연 프로젝트 번호.
5. 줄마다 `to_item` 에 계산한 값을 넘겨 `ProjectItem` 을 만들고 응답.

목록 한 번에 쿼리는 **4번** (개수 · 목록 · 멤버 수 · 연 적 있음) — 프로젝트가 10개든 50개든 같다.

5. 왜 이렇게

선택	이유
멤버 수 · opened 를 **모아서** 계산	줄마다 물으면 10개 목록에 쿼리 20번이 더 나간다 (N+1 문제). 템플릿 권장
`to_item` 은 값을 안 넘기면 직접 물어봄	상세 · 만들기 · 수정 응답은 한 개라 모을 필요가 없다 — 한 함수를 두 곳에서 같이 씀
정렬 뒤에 `project_id DESC`	수정 시각이 같은 프로젝트들의 순서가 쿼리마다 바뀌면 2쪽에 1쪽 것이 또 나오거나 빠질 수 있다
검색어의 `%` · `_` · `\` 를 escape	`LIKE` 에서 `%` = 아무 글자 여럿, `_` = 아무 글자 하나. 그대로 두면 `%` 검색이 모든 프로젝트를 돌려준다 (템플릿 권장)
남이 만들었지만 내가 멤버인 것도 포함	FR-123 "내가 참여한 프로젝트" — 역할은 `my_role` 로 구분 (Phase 6 초대 대비)
`opened` 는 **내** 세션만	다른 멤버가 열었어도 나는 아직 안 열었으면 Not opened (FR-130 은 사람마다)
없는 쪽은 빈 목록 (404 아님)	마지막 프로젝트를 지운 뒤 그 쪽에 머물러도 화면이 오류가 아니라 "없음"을 보여 줄 수 있다

6. 시험 (`tests/test_project_service.py` · 13개 추가, 모두 25개)

시험	확인한 것
`to_item` 계산	멤버 2명 → `member_count` 2, 남이 연 세션은 무시, 내 세션이 생기면 `opened` True
`to_item` 값 넘기기	넘긴 7 · True 를 그대로 씀, 프로젝트 칸 복사
정렬 3가지	updated · title(대소문자 무시) · created 순서
같은 값	수정 · 만든 시각이 같으면 나중에 만든 것 먼저
페이지	2개씩: 1쪽 2개 · 2쪽 1개 · 3쪽 빈 목록, `total` 은 늘 3
검색	`ALP` → `alpha` (대소문자 무시)
특수 문자	`%` → `100% Mix` 만, `_` → `my_song` 만
범위	남의 것 · 삭제된 것 제외, 멤버로 들어간 남의 프로젝트는 EDITOR · 멤버 2명으로 포함
opened 목록	내가 연 것만 True
빈 목록	프로젝트가 없으면 `items=[]`, `total=0`

전체 시험 463개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 59 — `create_project()` (프로젝트 만들기)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 새 프로젝트 · PRD 01 FR-122
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	58 `to_item` · `list_projects` · 60 `update_project`

1. 하는 일

새 프로젝트 모달의 값(56 에서 검사 끝)으로 **세 행을 한 번에** 저장한다.

표	넣는 행
`projects`	제목 · 설명 · BPM · 박자, `owner_id` = 나
`project_members`	(새 프로젝트, 나, **OWNER**)
`tracks`	이름 `Master`, 종류 **MASTER**, `order_index` 0

셋 중 하나라도 실패하면 **셋 다 저장되지 않는다** (rollback). 저장된 `Project` 를 돌려주면 라우터(62)가 `to_item` 으로 응답을 만든다.

보낸 값	저장 결과
`{"title": " New Song "}`	제목 `New Song`, 설명 없음, BPM 120.00, 4/4 + OWNER 행 + Master 트랙
`{"title": "A", "bpm": 128.5, "time_sig_num": 7, "time_sig_den": 8}`	BPM 128.50, 7/8
트랙 저장에서 DB 오류	프로젝트 · 멤버 행도 사라짐, 오류는 그대로 위로 (500)

2. 언제 쓰나

호출하는 곳	화면
`POST /api/v1/projects` 라우터 (62)	myProject.html 새 프로젝트 모달의 Create (P1-14)

트랜잭션: 여러 SQL 을 "모두 성공 또는 모두 취소" 한 묶음으로 처리하는 DB 기능.

- `db.add` — 세션에 올려 두기만 함 (아직 SQL 없음)
- `db.flush` — INSERT 를 DB 에 보냄. 하지만 **확정 전**이라 다른 연결에서는 안 보이고 되돌릴 수 있음
- `db.commit` — 묶음 전체를 확정
- `db.rollback` — 확정 전의 묶음 전체를 취소

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)

MASTER_TRACK_NAME = "Master"
```

`Track` · `MemberRole` · `TrackType` 을 이번에 추가했다. 이 함수는 `Session` · `Project` · `ProjectMember` · `Track` · `User` · `MemberRole` · `TrackType` · `ProjectCreateRequest` · `MASTER_TRACK_NAME` 을 쓴다.

함수

```
def create_project(db: Session, user: User, data: ProjectCreateRequest) -> Project:
    project = Project(
        owner_id=user.user_id,
        title=data.title,
        description=data.description,
        bpm=data.bpm,
        time_sig_num=data.time_sig_num,
        time_sig_den=data.time_sig_den,
    )
    try:
        db.add(project)
        db.flush()  # commit 전에 project_id 를 받기 위해 (INSERT 만 먼저 보냄)
        db.add(
            ProjectMember(
                project_id=project.project_id, user_id=user.user_id, role=MemberRole.OWNER
            )
        )
        # MASTER 는 화면에서 늘 맨 아래 고정이라 순서 번호는 쓰지 않음 (2026-10-06)
        db.add(
            Track(
                project_id=project.project_id,
                name=MASTER_TRACK_NAME,
                track_type=TrackType.MASTER,
                order_index=0,
            )
        )
        db.commit()
    except Exception:
        db.rollback()
```

```
        raise
    db.refresh(project)  # DB 가 채운 created_at · updated_at · bpm 등 다시 읽기
    return project
```

문법 풀이

import 문법	뜻
`from app.models.project import MemberRole`	역할 이름 목록 (OWNER · EDITOR · VIEWER). 글자를 직접 쓰는 것보다 오타가 나지 않음
`from app.models.track import TrackType`	트랙 종류 목록 (AUDIO · MIDI · MASTER · GROUP · RETURN)

식	뜻
`db.flush()` 뒤 `project.project_id`	INSERT 를 보내면 DB 가 만든 자동 번호가 객체에 채워짐 → 멤버 · 트랙 행에 씀
`try: ... except Exception: db.rollback(); raise`	어떤 오류든 먼저 취소하고, 같은 오류를 그대로 다시 던짐 (`raise` 만 쓰면 받은 오류 그대로)
`db.refresh(project)`	DB 에서 그 행을 다시 읽음 — `created_at` · `updated_at` 처럼 DB 가 채운 값을 받으려고

4. 흐름

1. 검사가 끝난 값으로 `Project` 객체를 만든다.
2. `add` → `flush` — 프로젝트 INSERT, 번호 받기.
3. 그 번호로 OWNER 멤버 · MASTER 트랙을 `add`.
4. `commit` — 남은 INSERT 두 개를 보내고 셋 다 확정.
5. 2~4 에서 오류 → `rollback` 으로 셋 다 취소 → 오류를 다시 던짐 (처리기가 500).
6. `refresh` → 돌려줌.

5. 왜 이렇게

선택	이유
세 행을 한 트랜잭션	프로젝트만 있고 OWNER 행이 없으면 `get_membership`(57)이 주인에게도 404 를 낸다 — 아무도 열 수 없는 프로젝트가 생김. MASTER 트랙이 없으면 시퀀서가 깨짐 (FR-122)
`flush` 로 번호를 먼저 받음	멤버 · 트랙 행에 `project_id` 가 필요하지만, 아직 확정하면 안 되므로 (템플릿 순서)
`except` 에서 `rollback` 후 `raise`	실패한 세션은 rollback 하기 전까지 다음 쿼리를 못 한다. 오류를 삼키지 않고 다시 던져야 처리기가 500 · Sentry 로 남긴다
`title.strip()` 하지 않음	스키마(56)가 이미 공백을 지움 (템플릿 TODO 의 `strip()` 은 필요 없어짐)
`MemberRole.OWNER` · `TrackType.MASTER`	글자 `"OWNER"` 대신 목록 이름 — 오타면 바로 오류
MASTER `order_index` 0	MASTER 는 트랙 목록 맨 아래 고정이라 순서 번호를 보지 않음 (2026-10-06 결정). 템플릿 값 그대로
프로젝트를 돌려줌 (`ProjectItem` 아님)	화면 모양으로 바꾸는 일은 `to_item` 이 맡음 — 만들기 · 수정 · 상세가 같은 응답 모양

6. 시험 (`tests/test_project_service.py` · 4개 추가, 모두 29개)

시험	확인한 것
세 행 저장	제목 공백 정리 · 설명 · BPM 128.50 · 7/8, `created_at` 등 DB 값, OWNER 행, Master 트랙 하나 (MASTER · 0)
기본값	제목만 → 설명 None · BPM 120.00 · 4/4
목록 · 응답	만든 뒤 목록에 OWNER · 멤버 1 · opened False 로 나옴
실패 시 취소	트랙 이름을 101자로 바꿔 트랙 INSERT 에서 DB 오류 → 프로젝트 · 멤버 행 0개, 먼저 확정한 회원은 그대로

전체 시험 467개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 60 — `update_project()` (제목 · 설명 수정)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 프로젝트 정보 수정 · PRD 01 FR-124
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	59 `create_project` · 61 `delete_project`

1. 하는 일

Edit details 모달에서 보낸 **제목 · 설명 중 보낸 칸만** 바꾼다. OWNER · EDITOR 만 할 수 있다.

보낸 값 (`PATCH` 본문)	결과
`{"title": " New Title "}`	제목 `New Title`, 설명 그대로, `updated_at` 갱신
`{"description": "new desc"}`	설명만 바뀜, 제목 그대로
`{"description": null}` · `""` · `" "`	설명 지움 (None)
`{}`	아무것도 바뀌지 않음, `updated_at` 도 그대로
`{"title": "T", "bpm": 200}`	제목만 바뀜 — BPM 은 이 API 로 못 바꿈 (시퀀서에서, Phase 2)
VIEWER 가 보냄	403 `"You don't have permission."`, 아무것도 안 바뀜
`{"title": null}` · `{"title": ""}`	422 — 여기까지 오지 않음 (스키마 56)

2. 언제 쓰나

호출하는 곳	화면
`PATCH /api/v1/projects/{id}` 라우터 (62) — `get_membership`(57) 으로 `project` · `member` 를 받은 뒤	myProject.html 카드의 Edit details 모달 Save (P1-14)

라우터는 돌려받은 프로젝트를 `to_item`(58) 으로 바꿔 응답한다 → 화면은 그 카드 한 장만 다시 그린다.

3. 코드

import (파일 맨 위 — 59 와 같음, 이번에 추가한 것 없음)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
```

```
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

이 함수는 `Session` · `Project` · `ProjectMember` · `MemberRole` · `ProjectUpdateRequest` 를 쓴다 (역할 검사는 57 `require_role`).

함수

```
def update_project(
    db: Session,
    project: Project,
    member: ProjectMember,
    data: ProjectUpdateRequest,
) -> Project:
    require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    changes = data.model_dump(exclude_unset=True)
    for name, value in changes.items():
        setattr(project, name, value)
    db.commit()
    db.refresh(project)  # 새 updated_at 읽기
    return project
```

문법 풀이

import 문법	뜻
(새 import 없음)	59 에서 추가한 `MemberRole` 을 역할 검사에 씀

식	뜻
`data.model_dump(exclude_unset=True)`	**본문에 실제로 있던 칸만** 사전으로. `{"title": "A"}` → `{"title": "A"}` (description 없음)
`for name, value in changes.items():`	사전의 (이름, 값) 쌍을 하나씩
`setattr(project, name, value)`	`project.title = value` 와 같음 — 칸 이름이 변수일 때
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	역할이 둘 중 하나가 아니면 403 (57)

`exclude_unset` 이 중요한 이유:

보낸 본문	`model_dump()`	`model_dump(exclude_unset=True)`
`{"title": "A"}`	`{"title": "A", "description": None}` → 설명이 **지워짐** ✗	`{"title": "A"}` ✓
`{"description": null}`	`{"title": None, "description": None}`	`{"description": None}` → 설명만 지움 ✓

4. 흐름

1. 역할 검사 — VIEWER 면 403, 아무것도 안 바꿈.

2. 보낸 칸만 꺼낸다.
3. 그 칸만 객체에 넣는다.
4. `commit` — 바뀐 칸만 `UPDATE`. 값이 바뀌었으면 DB 가 `updated_at` 을 지금으로.
5. `refresh` 로 새 `updated_at` 을 읽어 돌려줌.

5. 왜 이렇게

선택	이유
`exclude_unset=True`	보내지 않은 칸까지 None 으로 덮어쓰면 제목만 고쳐도 설명이 사라진다 (PATCH = 일부만 수정)
역할 검사를 **맨 앞**	VIEWER 의 요청은 객체를 건드리기 전에 끝낸다
`setattr` 반복	칸이 늘어도(Phase 2 의 키 · 스케일 등) 스키마에만 추가하면 됨. 스키마에 없는 칸(bpm 등)은 pydantic 이 버려서 들어올 수 없다
`updated_at` 을 직접 넣지 않음	DB 의 `ON UPDATE CURRENT_TIMESTAMP(3)` 가 맡음 — 빈 요청에는 UPDATE 가 없으니 시각도 그대로 (목록의 "최근 수정순"이 정확)
`try/rollback` 없음	한 행 UPDATE 하나라 반쯤 저장될 일이 없다. 실패하면 요청이 끝날 때 세션이 닫히며 취소됨 (59 는 세 행이라 필요)
`title.strip()` · null 검사 없음	스키마(56)가 이미 처리 (템플릿의 "title null → 422" 를 스키마로 옮김)

6. 시험 (`tests/test_project_service.py` · 9개 추가, 모두 38개)

시험	확인한 것
제목만	공백 정리된 제목, 설명 그대로, `updated_at` 이 10월 1일보다 뒤
설명만	제목 그대로
설명 지우기 3가지	null · `""` · `" "` → None
빈 본문	제목 · 설명 · `updated_at` 모두 그대로
스키마에 없는 칸	`bpm: 200` 은 무시 → 120.00
EDITOR	수정 가능
VIEWER	403, DB 의 제목 그대로

전체 시험 476개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 61 — `delete_project()` (프로젝트 삭제 · 소프트 삭제)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 프로젝트 삭제 · PRD 01 FR-125
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	60 `update_project` · 62 프로젝트 라우터

1. 하는 일

프로젝트 행을 **지우지 않고** `deleted_at` 에 지금 시각(UTC)을 적는다 (소프트 삭제). OWNER 만 할 수 있다.

상황	결과
OWNER 가 삭제	`deleted_at` 채움 → 목록에서 사라지고, 다시 열면 404
EDITOR · VIEWER 가 삭제	403 `"You don't have permission."`, 그대로
이미 삭제된 프로젝트를 또 삭제	404 — 여기까지 오지 않음 (`get_membership` 57)

표	삭제 뒤
`projects`	행 남음, `deleted_at` 만 채워짐
`project_members` · `tracks` (MASTER 등)	그대로 — 복구할 수 있게

2. 언제 쓰나

호출하는 곳	화면
`DELETE /api/v1/projects/{id}` 라우터 (62) — `get_membership`(57) 뒤	myProject.html 카드의 Delete → `confirmDialog` 확인 (P1-14)

소프트 삭제: 행을 `DELETE` 하지 않고 "지운 시각" 칸만 채워 두는 방식. 읽는 쪽(57 · 58)이 `deleted_at IS NULL` 인 것만 보므로 사용자에게는 지워진 것과 같다. 회원 탈퇴(P1-09)와 같은 방식.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
```

```
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from datetime import UTC, datetime` 줄을 이번에 추가했다. 이 함수는 `UTC` · `datetime` · `Session` · `Project` · `ProjectMember` · `MemberRole` 을 쓴다.

함수

```
def delete_project(db: Session, project: Project, member: ProjectMember) -> None:
    require_role(member, MemberRole.OWNER)
    project.deleted_at = datetime.now(UTC).replace(tzinfo=None)  # DB 는 UTC 를 시간대 없이 저장
    db.commit()
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime`	`datetime` = 날짜 · 시각, `UTC` = 세계 표준시 시간대

식	뜻
`datetime.now(UTC)`	지금 시각 (UTC, 시간대 정보 붙음)
`.replace(tzinfo=None)`	시간대 정보만 떼어 냄 — DB 의 `DATETIME` 칸은 시간대를 저장하지 않으므로 (값은 UTC 그대로)
`-> None`	돌려주는 값 없음 — 라우터는 204 (본문 없음) 로 응답

4. 흐름

1. 역할 검사 — OWNER 가 아니면 403.
2. `deleted_at` = 지금 (UTC).
3. `commit` → 끝. 이후 `get_membership` · `list_projects` 가 이 프로젝트를 보지 않는다.

5. 왜 이렇게

선택	이유
소프트 삭제	실수로 지운 것을 되살릴 수 있다. 트랙 · 클립 · 샘플 연결을 한꺼번에 지우는 큰 작업을 요청 중에 하지 않는다 (FR-125)
OWNER 만	협업자(Phase 6)가 남의 프로젝트를 통째로 없애지 못하게 (PRD 01 §5)
트랙 · 멤버는 그대로	복구할 때 그대로 돌아옴. 실제 파일 정리는 나중 정리 작업 (FR-137)
UTC 로 저장	다른 시각 칸(가입 · 탈퇴 · 문의)과 같은 규칙 — 화면에서 사용자 시간대로 바꿈
`refresh` 없음 · 돌려주는 값 없음	삭제 응답은 본문이 없으니 새 값을 읽을 필요가 없다
두 번 삭제는 신경 쓰지 않음	이미 삭제된 것은 57 이 404 로 막음

6. 시험 (`tests/test_project_service.py` · 4개 추가, 모두 42개)

시험	확인한 것
OWNER 삭제	행은 남고 `deleted_at` 이 지금 이후, 멤버 행 · MASTER 트랙 그대로
사라짐	목록에 남은 것만 (total 1), 지운 것은 `get_membership` 404, 남은 것은 그대로 열림
EDITOR · VIEWER	403 `FORBIDDEN`, `deleted_at` 그대로 None

전체 시험 480개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 62 — 프로젝트 라우터 5개 (`/api/v1/projects`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	P1-13 프로젝트 API — 주소 연결 · PRD 01 §7 · FR-122~126
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	61 `delete_project` · (P1-13 끝 — 개요 49, 다음 작업 P1-14 myProject.html JS)

1. 하는 일

요청 주소 5개를 서비스 함수(57~61)에 연결한다. 라우터는 **순서만 정하고**, 검사 · 저장은 서비스가 한다.

요청	성공	실패
`GET /api/v1/projects?query=&sort=&page=&size=`	200 `{items, total, page, size}`	401 로그인 안 함 · 422 틀린 값 (page 0, size 51, sort 모름, 검색어 101자)
`POST /api/v1/projects`	**201** `ProjectItem` (OWNER · 멤버 1 · opened false)	401 · 422 (56)
`GET /api/v1/projects/{id}`	200 `ProjectItem`	401 · **404** (없음 · 남의 것 · 삭제됨)
`PATCH /api/v1/projects/{id}`	200 `ProjectItem` (바뀐 값)	401 · 404 · **403** VIEWER · 422
`DELETE /api/v1/projects/{id}`	**204** 본문 없음	401 · 404 · **403** OWNER 아님

응답 예 (`POST`, 본문 `{"title": "My Song", "bpm": 128.5}`):

```
{"project_id": 12, "title": "My Song", "description": null, "bpm": "128.50",
 "time_sig_num": 4, "time_sig_den": 4, "music_key": null,
 "created_at": "2026-10-08T02:10:11.123000", "updated_at": "2026-10-08T02:10:11.123000",
 "my_role": "OWNER", "member_count": 1, "opened": false}
```

`bpm` 은 **글자**(`"128.50"`)로 온다 — 소수(Decimal)를 정확히 보내려는 pydantic 규칙. 화면(P1-14)에서 숫자가 필요하면 `Number(item.bpm)`.

2. 언제 쓰나

화면 (P1-14 myProject.html)	요청
페이지 열기 · 검색 · 정렬 · 페이지 버튼	`GET /projects`
새 프로젝트 모달 Create	`POST /projects` → 넓은 화면이면 시퀀서 창 열기 (FR-126 · 129)
카드 Edit details Save	`PATCH /projects/{id}`
카드 Delete (+ `confirmDialog`)	`DELETE /projects/{id}`
시퀀서가 열 때 제목 표시 (Phase 2)	`GET /projects/{id}`

`CurrentUser` 는 토큰을 검사해 회원을 넘겨주는 의존성(P1-03). 토큰이 없거나 만료되면 이 함수들은 불리지도 않고 401.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Query, status

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.project import (
    PROJECT_PAGE_SIZE_MAX,
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
from app.services import project_service
```

`from app.services import project_service` 줄을 이번에 추가했다. 나머지는 템플릿 그대로 — 주소 · 요청 · 응답 모양(`Query` · 스키마 · `status`)을 정하는 데 쓴다.

함수 (설명 주석 생략)

```
router = APIRouter()

NOT_FOUND = {404: {"model": ErrorResponse}}
FORBIDDEN = {403: {"model": ErrorResponse}}


@router.get("", response_model=ProjectListResponse)
def list_projects(
    user: CurrentUser,
    db: Db,
    query: Annotated[str, Query(max_length=100)] = "",
    sort: ProjectSort = ProjectSort.UPDATED,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=PROJECT_PAGE_SIZE_MAX)] = 10,
) -> ProjectListResponse:
    return project_service.list_projects(db, user, query.strip(), sort, page, size)


@router.post("", response_model=ProjectItem, status_code=status.HTTP_201_CREATED)
def create_project(data: ProjectCreateRequest, user: CurrentUser, db: Db) -> ProjectItem:
    project = project_service.create_project(db, user, data)
    # 방금 만든 프로젝트라 멤버는 나 하나 · 연 적 없음 → DB 에 다시 묻지 않음
    return project_service.to_item(db, project, project.members[0], member_count=1, opened=False)


@router.get("/{project_id}", response_model=ProjectItem, responses=NOT_FOUND)
```

```
def read_project(project_id: int, user: CurrentUser, db: Db) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    return project_service.to_item(db, project, member)


@router.patch(
    "/{project_id}",
    response_model=ProjectItem,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project(
    project_id: int,
    data: ProjectUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_project(db, project, member, data)
    return project_service.to_item(db, project, member)


@router.delete(
    "/{project_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def delete_project(project_id: int, user: CurrentUser, db: Db) -> None:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.delete_project(db, project, member)
```

문법 풀이

import 문법	뜻
`from app.services import project_service`	모듈째 가져옴 → `project_service.create_project` 처럼 씀. 라우터 함수 이름(`create_project`)과 서비스 함수 이름이 같아도 섞이지 않음

식	뜻
`@router.get("", response_model=...)`	이 함수 = `GET /api/v1/projects`. 돌려준 값을 응답 스키마 모양으로 걸러 JSON 으로
`status_code=status.HTTP_201_CREATED`	성공 코드 201 (새로 만듦). 삭제는 204 (본문 없음)
`responses={**NOT_FOUND, **FORBIDDEN}`	API 문서(/docs)에 404 · 403 응답 모양을 적음. 동작에는 영향 없음
`Annotated[int, Query(ge=1)] = 1`	주소의 `?page=` 값, 1 이상, 없으면 1. 틀리면 FastAPI 가 422
`query.strip()`	검색어 앞뒤 공백 제거 — `" gam "` → `"gam"`, `" "` → `""`(전체)
`project, member = ...get_membership(...)`	함수가 돌려준 두 값을 두 변수에 나눠 받음
`project.members[0]`	관계(P1-03)로 이 프로젝트의 멤버 목록 → 방금 넣은 OWNER 행 하나

4. 흐름

요청	순서
목록	검색어 공백 제거 → `list_projects` (58)
만들기	`create_project` (59) → `to_item` 에 멤버 1 · opened False 를 넘김
상세	`get_membership` (57) → `to_item`
수정	`get_membership` → `update_project` (60) → `to_item`
삭제	`get_membership` → `delete_project` (61) → 204

어느 단계에서든 `AppError` 가 나면 처리기(P1-03)가 `{"error": {"code", "message", "details"}}` 로 응답한다.

5. 왜 이렇게

선택	이유
라우터는 연결만	권한 · 검사 · 저장은 서비스에 — 서비스는 HTTP 없이 시험할 수 있고(57~61), 시퀀서 API(Phase 2)도 같은 함수를 씀
상세 · 수정 · 삭제 모두 `get_membership` 먼저	남의 프로젝트는 역할 검사 전에 404 로 끝남 (57 결정 A)
만들기 응답에 값 직접 넘김	방금 만든 프로젝트의 멤버 수(1) · opened(False)는 확실하므로 쿼리 두 번을 아낌 (템플릿)
수정 · 상세도 `ProjectItem`	목록 한 줄과 같은 모양 → 화면이 그 카드만 다시 그리면 됨
삭제는 204	돌려줄 내용이 없음. 화면은 성공이면 카드를 지우고 목록을 다시 읽음

6. 시험 (`tests/test_project_routes.py` · 12개 추가, 모두 13개 — 실제 앱 + MySQL)

시험	확인한 것
만들기	201, 응답 칸 12개, 제목 공백 정리, bpm `"128.50"`, 7/8, OWNER · 1 · false, DB 주인
만들기 422	공백 제목 → `VALIDATION_ERROR`
로그인 안 함	목록 · 만들기 · 상세 → 401
목록	이름순 2개씩 → `alpha` · `Beta`, total 3 / 검색 `" gam "` → `Gamma`
목록 422 (5가지)	page 0 · size 0 · size 51 · sort random · 검색어 101자
상세	내 것 200 (만들기 응답과 같음), 남의 것 · 없는 번호 404 "Project not found."
수정	OWNER 제목 → 설명 그대로 · 멤버 3, EDITOR 설명 지움, VIEWER 403, 제목 null 422, DB 그대로
삭제	EDITOR 403 → OWNER 204 본문 없음 → 상세 · 다시 삭제 404, 목록 total 0

전체 시험 492개 통과, `ruff` 검사 통과.